

Lerna Protocol

Factory Rights Settlement Profiles for Hydra/Cardano eUTxO

Arthur Zhang

8 June 2026

Abstract

Lerna is a Hydra-native discipline for representing many virtual-channel rights compactly inside a Hydra Head or an Interhead virtual Head, and for deterministically materialising those rights when optimistic updating stops. It is not a replacement for Hydra close, contest, or fanout. It sits beneath those mechanisms and supplies the missing inner rule: compact factory rights must be turned into ordinary Head UTxOs or claim UTxOs before they can be safely exposed to the outer settlement layer. The paper presents one protocol with two settlement profiles. The Head-local factory profile keeps factory rights inside the Head ledger and relies on authorised checkpointing or materialisation before Hydra contestation expires. The claim-materialisation profile makes that repayment explicit through a script-controlled claim UTxO, an Aiken spending validator, deterministic off-chain transition checks, real Hydra/Cardano devnet evidence, and an executable security-theorem gate.

The design is intentionally bounded. Lerna does not compete with Interhead Hydra’s virtualisation of Heads across Heads, nor with Hydrozoa’s broader execution/custody separation and deterministic outer rollout. Lerna uses those works as boundary conditions: Interhead explains where virtual Heads may live; Hydrozoa shows why compact off-chain state must be rolled out deterministically when custody is at stake; Hydra Head supplies the close/contest/fanout substrate and the practical pressure that makes compactness valuable. Lerna’s originality is narrower and more internal: an inner factory-rights discipline, deterministic inner materialisation before outer fanout or rollout, and factory-local non-interference plus release receipts. The implementation-backed claim-materialisation profile is conditional on Hydra contestability, Cardano ledger validation of submitted transactions, proof availability, watcher/operator liveness, and materialisation capacity inside the committed budget. If the Head-local liveness assumption is unacceptable, the claim-materialisation profile is the appropriate safety envelope.

Keywords: Cardano, Hydra, eUTxO, claim UTxO, Aiken, Plutus, virtual channels, state channels, non-interference, native assets, release receipts.

Contents

1	Introduction	1
2	Layering	2
2.1	Lerna below Interhead: from virtual Heads to virtual rights	2
2.2	Inner rollout before outer rollout: Lerna and Hydrozoa	2
2.3	Why compact factory state matters for Hydra fanout	2
2.4	Compatibility across host ledgers	3
2.5	What Lerna deliberately does not take from each prior work	3
3	System and Threat Model	4
3.1	Actors	4
3.2	Adversary	4
3.3	Assumptions	4
4	Factory and Claim State	5
5	Materialisation Transition	5
5.1	Admission	6
5.2	Authorisation	6
5.3	Conservation	6
5.4	Release Receipts	6
5.5	Continuation	6
6	Validator Enforcement	7
7	Implementation Evidence for the Claim-Materialisation Profile	7
8	Evidence and Evaluation	8
9	Limitations	9
10	Conclusion	9
A	Reproducibility Checklist	10

1 Introduction

Hydra Head gives a fixed participant set a Cardano-like off-chain ledger: participants deposit UTxOs, sign snapshots, close with a snapshot when necessary, contest stale closes, and fan out the resolved UTxO set to L1 [1, 2]. This is already a powerful settlement substrate. A virtual-channel factory layered inside or near a Head should therefore avoid inventing an unmeasured parallel dispute game. Its hard problem is narrower and more operational: when compact rights are no longer being updated optimistically, what exact inner object or transition discipline turns them back into ordinary ledger claims without double release, stale-state settlement, or silent asset loss?

Lerna answers that question as a factory-rights protocol rather than as a roadmap. In the Head-local factory profile, compact rights remain inside the Head or virtual Head and are made safe by checkpointing or materialising the latest Lerna state into the Hydra snapshot stream before the outer contestation deadline expires. In the claim-materialisation profile, the same repayment is made explicit by a script-controlled claim UTxO carrying unresolved value, latest-state metadata, authorisation policy, release-receipt state, proof-availability commitments, and materialisation bounds. Settlement spends the claim through a compiled Aiken validator. A final settlement consumes all unresolved claim value. A partial settlement creates a continuation claim UTxO with an inline datum and a strictly advanced settled-right frontier.

This paper is written to keep those two profiles in one theory. The claim-materialisation implementation is not presented as a Hydra replacement, a Hydrozoa competitor, or an “eltoo for Cardano”. It is implementation evidence for the stronger materialisation profile, checked by the repository at commit 66ff116. The acceptance commands are:

```
npm test
npm run verify:security-theorem
npm run check:devnet
npm run check:profiles
```

Contributions. The paper makes three concrete contributions.

C1. Inner factory rights discipline.

Lerna defines the compact object that lives below Hydra fanout: a factory-local rights state binding factory identity, Hydra Head identity, network id, settlement epoch, latest-state number, unresolved value, signer policy, release-receipt root, proof-availability root, and materialisation bounds. The same discipline can be represented as Head UTxOs in the Head-local profile or as a claim datum in the claim-materialisation profile.

C2. Deterministic inner materialisation before outer fanout or rollout.

Lerna makes compactness payable. Before the outer Hydra fanout, Interhead movement, or Hydrozoa-style rollout is relied upon for custody, compact factory rights must be deterministically expanded into ordinary ledger outputs or into a continuation object with an explicitly bounded remaining value.

C3. Factory-local non-interference plus release receipts.

Lerna separates authorised partial settlement from unauthorised interference. Reduced-signature materialisation is admissible only with a factory-local frame proof that untouched owners remain untouched. Each released right also receives a deterministic receipt, so the protocol has both a non-interference argument and a nullifier against double release.

2 Layering

Lerna stays inside the Cardano/Hydra settlement vocabulary. Cardano L1 supplies eUTxO accounting, validators, inline datums, reference scripts, native assets, validity intervals, and min-ADA rules [4]. Hydra supplies the outer Head lifecycle: open, snapshot, close, contest, and fanout. Lerna supplies the inner factory discipline used when many compact virtual-channel rights must be materialised into ordinary ledger outputs.

2.1 Lerna below Interhead: from virtual Heads to virtual rights

Interhead virtualises Hydra Heads across Heads: it studies how two Heads and their intermediaries can move a virtual Head state between Head contexts while preserving adjudication conditions [5]. Lerna operates at the next smaller granularity. It does not create a virtual Head and does not specify cross-Head transport. Instead, it virtualises rights and channels inside one physical Head, or inside a virtual Head supplied by an Interhead construction.

The distinction is important. A virtual Head still eventually exposes a Head state to Hydra-like closing and settlement. Lerna asks what happens inside that state when a factory has compressed many bilateral or application-specific rights into a small number of Head UTxOs. The answer is factory-local: the latest Lerna state must bind the Head id, factory id, network id, settlement epoch, unresolved value, owners, receipts, and materialisation budget. That state may live as compact Head-local accounting, or it may be represented by a claim UTxO when explicit ledger settlement is required. In both cases, Lerna is below Interhead: Interhead says where the virtual Head may be; Lerna says how rights inside that Head are released.

2.2 Inner rollout before outer rollout: Lerna and Hydrozoa

Hydrozoa separates execution from custody and gives deterministic rollout or fallback procedures for turning L2 ledger effects into L1 transactions when large withdrawal sets must be paid out [6]. Lerna borrows the design ethic, not the ledger architecture. It treats compact factory rights as an inner state that must be deterministically rolled out before the outer system is asked to carry custody.

This creates a simple ordering rule. First, materialise the compact Lerna factory state into ordinary Head UTxOs or into a bounded continuation claim. Only then rely on Hydra fanout, Interhead movement, or Hydrozoa-style rollout for the outer layer. Without this inner rollout, the outer layer sees a small UTxO whose datum names a compressed factory, not the individual rights that users expect to receive. Lerna's claim-materialisation profile is the measured implementation of this rule: it turns a compact claim into selected settlement outputs, release receipts, and, if necessary, one continuation claim whose remaining value and frontier are explicit.

2.3 Why compact factory state matters for Hydra fanout

Hydra fanout is the final exposure point of a Head state. The Hydra documentation records practical constraints around Head finalisation: large or complex UTxO sets can exceed fanout capacity, and tokens minted but not burned inside an open Head can prevent finalisation [3]. These are not abstract inconveniences for factories. A naive virtual-channel factory that keeps every right as a separate Head UTxO can create fanout pressure exactly when the system is least able to negotiate new structure. A factory that mints control tokens internally and leaves them unresolved can create a finalisation hazard.

Lerna's compactness is a direct response to that pressure. During ordinary operation, the Head carries a small factory object rather than a fanout-sized collection of user outputs. Compactness, however, is debt. Before the Head is closed, contested, fanned out, or rolled into

another custody context, Lerna must repay the debt by deterministic materialisation: selected rights become ordinary outputs, release receipts advance, remaining rights are represented by one bounded continuation, and the value equation is checked exactly over lovelace and native assets. Compact state is therefore acceptable only because the protocol specifies how it becomes non-compact again.

2.4 Compatibility with physical Heads, Interhead virtual Heads, and Hydrozoa-style ledgers

Lerna is compatible with three host settings.

Physical Hydra Head.

The Head-local profile stores compact factory state directly in the Head ledger. The claim-materialisation profile introduces a claim UTxO when the factory needs script-controlled release, reduced-signature settlement, or multi-batch continuation before outer fanout.

Interhead virtual Head.

Interhead may supply the virtual Head context. Lerna does not alter the Interhead conversion or punish phases. It binds its own rights state to the virtual Head's identity and requires inner materialisation before that state is treated as settled across Head boundaries.

Hydrozoa-style ledger.

A Hydrozoa-style system may provide deterministic outer rollout and rule-based fallback. Lerna can be used as an inner rights factory whose compact state is expanded into ledger outputs before the Hydrozoa rollout algorithm is asked to distribute custody.

The compatibility claim is deliberately modest. Lerna assumes the host system has a notion of ledger outputs, value conservation, domain separation, deadlines, and a way to submit deterministic materialisation transactions. It does not require the host system to adopt Lerna's claim validator, and it does not require Lerna to adopt the host system's full dispute game.

2.5 What Lerna deliberately does not take from each prior work

From Hydra Head.

Lerna takes the Head lifecycle as substrate. It does not replace open, close, contest, or fanout, and it cannot rescue a stale Head close that no authorised actor contests in time.

From Interhead.

Lerna takes the idea that Head contexts can themselves be virtualised. It does not take over Interhead's cross-Head conversion protocol or intermediary aggregation. Its unit is a right inside a Head, not a Head across Heads.

From Hydrozoa.

Lerna takes the discipline of deterministic rollout when compact L2 state must be exposed to custody. It does not adopt Hydrozoa's execution/custody split as its own ledger model and is not a competing state-channel architecture.

From eltoo, factories, and Perun.

Lerna takes the lesson that latest-state replacement and factory adjudication should avoid punitive games when possible [7, 8, 9]. It does not

claim to be an eltoo construction for Cardano, and it does not import Bitcoin-style update transactions into the eUTxO setting.

3 System and Threat Model

3.1 Actors

Head participant.

A Hydra party able to run a Hydra node, sign snapshots, close, and contest.

Virtual participant.

An application endpoint whose right is represented in Lerna factory state. A virtual participant may or may not be a Head participant.

Operator.

An application actor that can coordinate batching, submit transactions, or fund ordinary fees. The operator does not gain authority to redirect factory value.

Watcher.

An actor that observes Head lifecycle events and Lerna evidence, and can cause fresh evidence or settlement transactions to be submitted through an authorised path.

3.2 Adversary

The adversary may corrupt participants, operators, watchers, or Head participants; withhold signatures; provide stale latest-state evidence; propose same-number conflicting states; withhold proof bodies; try replay across domains, networks, Heads, factories, or epochs; create outputs below min-ADA; over-release lovelace or native assets; reorder multi-batch settlement; mutate continuation datums; leave minted control tokens unresolved; force large fanout sets; or tamper with local evidence files used by implementation acceptance.

3.3 Assumptions

Assumption 1 (Hydra contestability). *If the Head is closed with stale state, at least one honest actor can observe the close and contest with newer valid Head evidence before the contestation deadline.*

Assumption 2 (Cardano ledger validation). *Submitted materialisation, claim-creation, claim-settlement, and continuation transactions are validated by the Cardano ledger according to the observed devnet rules.*

Assumption 3 (Head-local checkpoint or materialisation). *When the Head-local factory profile is used, safety requires at least one authorised Head participant, operator, or watcher to checkpoint or materialise the latest Lerna state into a Hydra snapshot before the relevant contestation period expires. If this liveness assumption is unacceptable for the application or deployment environment, the design must use the claim-materialisation profile rather than relying on Head-local accounting alone.*

Assumption 4 (Proof availability). *Proof bodies committed by the proof witness root are available to the verifier before materialisation is accepted.*

Assumption 5 (Watcher and operator liveness). *At least one actor with sufficient evidence and authority can submit accepted materialisation plans within the relevant Head and ledger deadlines. This is a liveness assumption, not a claim that Lerna can force offline parties to act.*

Assumption 6 (Bounded materialisation). *The number of outputs, proof bytes, execution units, and continuation value fit within the budget committed in the factory state or claim datum.*

4 Factory and Claim State

Definition 1 (Settlement scope). *A Lerna settlement scope is the tuple*

$$\Sigma = (\text{domain}, \text{networkId}, \text{hydraHeadId}, \text{factoryId}, \text{claimEpoch}).$$

Every latest-state header, transition, receipt, proof-availability commitment, and non-interference proof is interpreted only inside this scope.

The word `claimEpoch` is used here as a settlement epoch, not as a release label. In the Head-local profile, the same scope binds compact factory state inside the Head ledger. In the claim-materialisation profile, the scope is carried by the claim datum and checked by the Aiken validator.

The implementation-backed profile uses the following domain separators:

Domain	Purpose
<code>Lerna:ClaimUTx0:LatestStateHeader</code>	latest-state and transition binding
<code>Lerna:ClaimUTx0:ReleaseReceipt</code>	release receipt nullifiers
<code>Lerna:ClaimUTx0:ProofAvailability</code>	proof witness package binding
<code>Lerna:ClaimUTx0:NonInterferenceProof</code>	reduced-signature frame proof

Definition 2 (Claim datum). *A claim datum C is the ledger representation of compact factory rights in the claim-materialisation profile. It contains:*

$$C = (\text{factoryId}, \text{hydraHeadId}, \text{claimEpoch}, \text{networkId}, \text{domain}, \text{latestStateNumber}, \text{stateCommitment}, \text{releaseReceiptRoot}, \text{nonInterferenceRoot}, \text{authorisationMode}, \text{unresolvedLovelace}, \text{unresolvedAssets}, \text{minAdaFloor}, \text{requiredSigners}, \text{settledRightCount}, \text{lastSettledRightId}, \text{maxOutputCount}, \text{maxProofSizeBytes}, \text{maxExecutionMemory}, \text{maxExecutionSteps}, \text{proofAvailabilityMode}, \text{proofWitnessRoot}).$$

The state commitment is recomputed from length-prefixed canonical fields, not accepted as an independent assertion.

The claim value is:

$$\text{Value}(C) = \text{lovelace}(C.\text{unresolvedLovelace}) + C.\text{unresolvedAssets}.$$

The Head-local profile uses the same conceptual fields, but their enforcement comes from the Head snapshot discipline and the authorised checkpoint/materialisation assumption rather than from spending a claim UTxO on L1. This is the main reason the paper treats the two settlement profiles as safety envelopes, not as software releases.

5 Materialisation Transition

A materialisation transition selects a canonical batch of rights $R = (r_1, \dots, r_k)$ with matching settlement outputs $O = (o_1, \dots, o_k)$. In the claim-materialisation profile, the transition spends the current claim UTxO, provides a redeemer D , an authorised signer set S , and produces either a final zero-claim result or a continuation claim datum C' . In the Head-local profile, the same transition discipline must be reflected in the Head snapshot before the outer Hydra contestation window can be relied upon.

5.1 Admission

The transition is admitted only if:

1. $D.\text{nextStateNumber} > C.\text{latestStateNumber}$.
2. The domain, network id, Head id, factory id, and claim epoch in the redeemer match the datum.
3. $k > 0$ and $k \leq C.\text{maxOutputCount}$.
4. Every selected output is at or above $C.\text{minAdaFloor}$.
5. Proof size and script-cost evidence are within the committed bounds.
6. The batch starts strictly after $C.\text{lastSettledRightId}$ in the canonical settled-right order.

5.2 Authorisation

Lerna has two implemented authorisation modes. In full authorisation, the settlement evidence must contain exactly the datum-bound signer set. In reduced-signature mode, the authorised signers must be an exact proper subset and the non-interference witness must show that all materialised outputs belong to signed owners and that the untouched frame is preserved.

The implementation evidence includes reduced-signature settlement on a real devnet run. This matters because reduced-signature settlement is a common source of accidental over-claiming: without a frame proof, an output for an unsigned owner could be smuggled into a partial settlement.

5.3 Conservation

Let M be the value materialised by the selected settlement outputs and *Remainder* be the value recorded in the continuation claim, or zero for a final settlement. The validator and deterministic model require:

$$\text{Value}(C) = M + \text{Remainder}.$$

This equality is exact over lovelace and native assets. The native-asset profile is therefore not merely a text extension; it is a profile-matrix run whose real devnet evidence must pass.

5.4 Release Receipts

For each materialised right r_i and selected output o_i , Lerna emits a deterministic receipt:

$$\text{Receipt}_i = \text{H}(\text{Lerna:ClaimUTxO:ReleaseReceipt}, \Sigma, r_i.\text{rightId}, \text{desc}(o_i), i).$$

The transition advances the receipt root from the previous root to a new root covering the settled batch. A later settlement cannot materialise the same right prefix again because the canonical frontier and receipt root must both advance.

5.5 Continuation

If unresolved value remains, exactly one continuing output must return to the claim script address with an inline datum C' . That datum must preserve the scope, signer policy, non-interference root, materialisation budget, proof availability mode, and proof witness root, while advancing the latest-state number, receipt root, settled-right count, last settled right id, unresolved value, and state commitment. If no unresolved value remains, there must be no continuing claim output.

6 Validator Enforcement

The Aiken validator `validators/lerna_claim.ak` enforces the settlement rules at the script boundary. Its key checks are:

Check family	Validator responsibility
Input and continuation	Resolve the own input, compare its value to claim value, require either no continuation for final settlement or exactly one inline-datum continuation.
Identity and domain	Bind datum and redeemer to the same factory, Head, network, epoch, and latest-state domain.
Latest state	Require the old commitment to match the datum and the next state number to be strictly greater.
State commitment	Recompute the current and next state commitments from canonical fields.
Receipt progress	Recompute the expected release-receipt root and require it to advance.
Authorisation	Check exact full signers or exact reduced-signature non-interference signers.
Conservation	Require input claim value to equal selected outputs plus continuation value across lovelace and native assets.
Output descriptors	Bind selected output indices, payment key hashes, lovelace, and asset bundles to the redeemer descriptors.
Admission bounds	Enforce positive numeric bounds, output-count bounds, min-ADA policy, and continuation-value safety.

These checks are intentionally redundant with the deterministic off-chain model. The model explains and generates expected transitions; the validator decides whether an on-chain spend is admissible; the real devnet verifier checks that the claimed local evidence refers to the actual generated artifacts.

7 Implementation Evidence for the Claim-Materialisation Profile

Theorem 1 (Implementation security theorem). *For any implementation-accepted claim settlement, if the Hydra/Cardano devnet evidence, local files, profile matrix, and theorem gate all pass, then the settlement:*

1. *spends the script-controlled claim UTxO through the compiled claim validator with matching inline datum, redeemer, and artifacts;*
2. *binds latest-state evidence to one domain, network id, Head id, factory id, and claim epoch;*
3. *strictly supersedes the previous latest-state number;*
4. *preserves recomputable datum, transition, and continuation commitments;*
5. *uses exact full authorisation or reduced-signature non-interference;*

6. *conserves lovelace and native assets across selected outputs and continuation state;*
7. *emits deterministic release receipts and advances the settled-right frontier;*
8. *respects committed output-count, proof-size, execution-unit, and min-ADA bounds;*
9. *binds proof availability before materialisation;*
10. *accepts only hash-matching local ledger, transaction, artifact, script-cost, and Hydra lifecycle evidence; and*
11. *reaches profile acceptance only when no unresolved claim value remains.*

The theorem is not a hand-waved claim. It is encoded in the repository’s security-theorem verifier as named obligations. The verifier fails unless each obligation is covered by the deterministic model, validator source keywords, strict real devnet checks, local file-hash verification, profile-matrix evidence, and negative tests.

Obligation id	Meaning
claim-utxo-ledger-enforcement	Claim spend uses the compiled validator and matching artifacts
latest-state-supersession	Same-scope settlement strictly advances state number
state-commitment-integrity	Commitments are recomputable from canonical fields
exact-authorisation	Full or reduced signer evidence matches the datum policy
non-interference-frame	Reduced settlement affects only signed owners
asset-conservation	Lovelace and native assets are exactly conserved
release-receipt-nullifier	Materialised rights cannot be claimed twice
multi-batch-progress	Partial batches advance a canonical frontier
materialisation-admission	Output count, proof size, costs, and min-ADA are bounded
proof-availability	Witness availability is bound before materialisation
ledger-evidence-authenticity	Local files and ledger observations hash-match the report
finality-of-accepted-report	Final acceptance leaves no unresolved claim value

8 Evidence and Evaluation

The implementation repository contains deterministic evidence, real Hydra/Cardano devnet evidence, and a profile matrix for the claim-materialisation profile. Implementation acceptance uses four commands:

```
npm test
npm run verify:security-theorem
npm run check:devnet
npm run check:profiles
```

`npm test` runs the Node test suite, including negative tests for stale state numbers, signer mismatches, conservation failures, receipt mismatches, output descriptor mutations, and local evidence mismatches. `npm run verify:security-theorem` checks the obligation coverage. `npm run check:devnet` probes the toolchain, builds the Aiken blueprint, runs deterministic claim

evidence, runs the real Hydra/Cardano devnet flow when available, refreshes deterministic evidence with the real Head scope, verifies the real evidence, and asserts acceptance. `npm run check:profiles` requires the reduced-signature, native-asset, and multi-batch profiles to pass real devnet verification.

Profile	Authorisation	Native assets	Batching
reduced-signature	reduced non-interference	none	final settlement
native-asset	reduced non-interference	exact conservation	final settlement
multi-batch	reduced non-interference	none	two batches

As of the evidence generated on 6 June 2026, all three profiles report `passFailStatus = passed`, `realDevnetPassFailStatus = passed`, and `noUnresolvedClaimsAfterSettlement = true`.

9 Limitations

The unified profile framing removes roadmap language; it does not remove the safety boundary. Lerna remains bounded in the following ways.

1. The claim-materialisation implementation is local devnet evidence, not a mainnet deployment.
2. The theorem assumes Cardano ledger validation and Hydra lifecycle observations behave as recorded.
3. The Head-local profile is safe only under the authorised checkpoint/materialisation assumption stated above. If that assumption is too strong, the application must use claim-materialisation rather than compact Head-local accounting alone.
4. Proof bodies committed by the proof witness root must be available to the verifier.
5. Watcher/operator liveness remains a real deployment assumption.
6. Materialisation is safe only within the committed output-count, proof, execution-unit, and min-ADA budget.
7. Dynamic membership, arbitrary application state, policy-level mint/burn logic, and cross-Head settlement composition are outside the accepted claim-materialisation profile.

10 Conclusion

Lerna’s position is precise when it is described through settlement profiles. Interhead virtualises Heads across Heads; Lerna virtualises rights and channels inside a Head or virtual Head. Hydrozoa shows why execution and custody should be separated and why rollout must be deterministic; Lerna applies that ethic to the inner materialisation of compact factory rights. Hydra Head supplies the close/contest/fanout substrate and the practical constraints that make compactness valuable; Lerna repays compactness through deterministic materialisation, release receipts, non-interference checks, and exact value conservation.

The claim-materialisation implementation is therefore evidence for one strong settlement profile, not the whole identity of the protocol. The Head-local factory profile remains useful when its liveness assumption is acceptable. The claim-materialisation profile is appropriate when that assumption is too strong and explicit ledger settlement is required. The remaining work is to keep tightening the theorem boundary: expand independent review of the Aiken validator, reproduce devnet evidence on fresh infrastructure, measure mainnet era limits, and state every deployment assumption at the same precision as the code checks it.

A Reproducibility Checklist

1. Check the implementation commit: 66ff116.
2. Confirm the paper is using the intended settlement profile: Head-local factory or claim-materialisation.
3. Run `npm test`.
4. Run `npm run verify:security-theorem`.
5. Run `npm run check:devnet`.
6. Run `npm run check:profiles`.
7. Inspect `evidence/profile-matrix/profile-matrix.json`.
8. Inspect `docs/LERNA_SECURITY_THEOREM.md` and confirm every obligation id appears in the theorem gate.

References

- [1] S. Chakravarty, M. D. Coretti, M. Fitzi, P. Gazi, P. Kant, A. Kiayias, and A. Russell. Hydra: Fast Isomorphic State Channels. IACR Cryptology ePrint Archive, Report 2020/299. <https://eprint.iacr.org/2020/299.pdf>.
- [2] Cardano Scaling. Hydra Head protocol documentation. <https://hydra.family/head-protocol/docs/protocol-overview>.
- [3] Cardano Scaling. Hydra known issues and limitations. Consulted 8 June 2026. <https://hydra.family/head-protocol/docs/known-issues>.
- [4] Cardano Documentation. The Extended UTxO accounting model. <https://docs.cardano.org/about-cardano/learn/eutxo-explainer/>.
- [5] M. Jourenko, M. Larangeira, and K. Tanaka. Interhead Hydra: Two Heads are Better than One. IACR Cryptology ePrint Archive, Report 2021/1188. <https://eprint.iacr.org/2021/1188.pdf>.
- [6] G. Flerovsky, I. Rodionov, and P. Dragos. Hydrozoa: Lightweight state channels for Cardano. Technical specification working draft, 7 November 2025. <https://cardano-hydrozoa.github.io/hydrozoa/hydrozoa.pdf>.
- [7] C. Decker, R. Russell, and O. Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. <https://blockstream.com/eltoo.pdf>.
- [8] C. Burchert, C. Decker, and R. Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file/49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.
- [9] S. Dziembowski, S. Faust, and K. Hostakova. General State Channel Networks. ACM CCS 2018.
- [10] a19q3. Lerna claim-UTxO implementation repository. Commit 66ff116, consulted 8 June 2026. <https://github.com/a19q3/Lerna>.